GraphitePDF

v 0.2.0

A pure Rust PDF rendering engine

Layout - Composition - Rendering Pipelines

FOUR-PAGE SHOWCASE

Template - Layout - Render

This document is produced entirely in Rust by the GraphitePDF engine and serves as a live demonstration of its four-layer pipeline.

-> pdf!, styles!, and stylesheet! macros compile JSX-like markup to typed builder calls.
-> The layout engine resolves text flow, SVG, images, and math equations.
-> The render engine serialises the safe layout document into a PDF file.
-> Each crate is independently versioned and usable at any abstraction level.

Named Styles and Font Families

Helvetica (sans-serif)

The default font family for headings and UI text. Clean, geometric, and highly legible at small sizes.

ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789

Times Roman (serif)

The classic serif typeface. Ideal for long-form body text, citations, and academic typesetting.

Timeless elegance - serifs that guide the reader through long paragraphs.

Courier (monospace)

Fixed-width for code listings, terminal output, and technical identifiers.

```
fn render() -> Result<(), Box<dyn Error>> { render_to_file(&doc, &path) }
```

stylesheet! macro - compile-time design tokens

The `ds` struct on this page was produced by: stylesheet! { .subheading => { ... }, .body => { ... } }.
Each named field is a plain LayoutStyle, so it integrates seamlessly with the Node builder API.
Used above: ds.subheading and ds.body - both zero-cost at runtime, resolved at compile time.

Equations and Rich Typography

graphitepdf-math - LaTeX to SVG to PDF

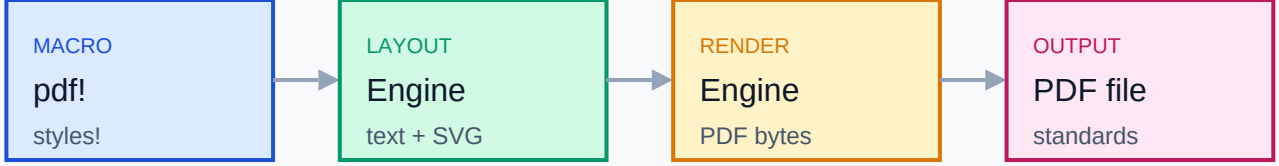
Equations are written in LaTeX and rendered to SVG by the math crate before compositing into the page.
Pass MathOptions to control display vs inline mode, colour, and debug SVG output.

Euler's identity - considered the most beautiful equation in mathematics.

Gaussian integral - the area under a Gaussian curve over the entire real line.

TextKit + Font Crate This node is constructed directly with graphitepdf-textkit spans. Each span carries its own FontDescriptor and size; the layout engine merges them into a single attributed string and runs the text engine to produce a measured TextLayout before page composition.

Image assets carry data-uri, local path, or remote URL sources. The image crate resolves them at render time and embeds them in the PDF content stream.



Template macros expand -> layout resolves nodes -> render serialises -> valid PDF output

Crate Ecosystem

graphitepdf	Facade - re-exports the full public API surface
graphitepdf-layout	Layout engine - document model and sizing pipeline
graphitepdf-render	Render engine - safe layout document to PDF bytes
graphitepdf-template	Runtime surface for the pdf! macro DSL
graphitepdf-template-macros	Proc macros - pdf!, styles!, stylesheet!
graphitepdf-svg	SVG parser and node model
graphitepdf-math	Math equation renderer (LaTeX to SVG to PDF)
graphitepdf-textkit	Text layout and attributed string engine
graphitepdf-font	Font descriptors and standard PDF font registry
graphitepdf-image	Raster and data-URI image source resolution
graphitepdf-primitives	Value types: Color, Pt, Size, Bounds
graphitepdf-stylesheet	CSS-style property sheet and resolver
graphitepdf-kit	Low-level PDF byte-stream builder (kit API)

Rendering Pipeline

1. Template macros expand JSX-like markup into typed builder calls at compile time.
2. The layout engine resolves sizes, flows text, and composites SVG and math nodes.
3. The render engine walks the safe layout document and emits PDF content streams.
4. render_to_file() writes the complete, standards-compliant PDF to disk.